

gemstone-rs Codegen Guide

Generating checked-in Rust wrappers for GemStone classes



Generated from the Markdown sources in `gemstone-rs/docs`.

Contents

Rust Codegen

Install

Config Format

Method Metadata

Mapped Structs

Commands

Generated Shape

Generated Wrapper App

Generated Object Mapping

VS Code Workflow

Rust Codegen

`gemstone-rs` codegen creates small Rust wrapper structs around GemStone OOPs. The goal is to move repeated selector strings into checked-in generated code while keeping the mapping reviewable.

Install

```
cargo install gemstone-rs-cli
gemstone-rs codegen --help
```

From a checkout:

```
cargo run -p gemstone-rs-cli -- codegen preview examples/codegen/gemstone-rs.codegen
```

Config Format

The config is intentionally line-oriented:

```
output = examples/codegen/generated/gemstone_wrappers.rs
class = Object
method = Object>>printString | return=String | doc=Return the receiver printString.
method = Object>>class
```

Use `Dictionary:ClassName` when a class should resolve from a specific dictionary:

```
class = UserGlobals:OkzBooking
method = UserGlobals:OkzBooking>>findById: | args=id | return=Oop | doc=Find a
booking by id.
```

Class-side references use `class`:

```
class = UserGlobals:OkzBooking class
method = UserGlobals:OkzBooking class>>findById: | args=id | return=Oop
```

Method Metadata

Option	Example	Purpose
<code>args</code>	<code>args=id,user</code>	Names generated Rust function arguments.
<code>return</code>	<code>return=String</code>	Generates a typed return helper instead of <code>Value</code> .
<code>doc</code>	<code>doc=Find by id.</code>	Writes a Rust doc comment above the generated method.

When `args` is omitted, codegen now infers useful argument names from selector keywords. For example `at:put:` becomes `at, put`, and `withCustomer:amount:` becomes `customer, amount`. Provide explicit `args=...` when the selector keywords are not good Rust argument names.

Supported return types:

Config value	Rust return
<code>Value</code>	<code>gemstone_rs::Value</code>
<code>String</code>	<code>String</code>
<code>SmallInt</code>	<code>i64</code>
<code>Bool</code>	<code>bool</code>
<code>Oop</code>	<code>gemstone_rs::Oop</code>

Mapped Structs

Codegen can also emit typed `BridgeMapped` structs for values stored under `BridgeRoot`:

```
mapped = BookingDraft | doc=A typed Rust payload stored under BridgeRoot.
field = BookingDraft.name | type=String | key=name
field = BookingDraft.amount | type=SmallInt | key=amount | key_type=Symbol
field = BookingDraft.currency | type=String | key=currency
field = BookingDraft.tags | type=Vec<String> | key=tags
```

Supported field types are `String`, `SmallInt`, `Bool`, `Oop`, `Mapped<OtherStruct>`, and `Vec<T>`.

Field options:

Option	Example	Purpose
<code>type</code>	<code>type=Vec<String></code>	Rust/GemStone field conversion.
<code>key</code>	<code>key=amount</code>	GemStone dictionary key.
<code>key_type</code>	<code>key_type=Symbol</code>	Use string keys or symbol keys explicitly.
<code>doc</code>	<code>doc=Booking amount.</code>	Writes a Rust doc comment above the field.

Commands

Create a starter config:

```
gemstone-rs codegen init gemstone-rs.codegen
```

Generate a config from a live stone:

```
gemstone-rs codegen discover gemstone-rs.codegen Object
gemstone-rs codegen discover gemstone-rs.codegen UserGlobals:OkzBooking
gemstone-rs codegen discover-mapping gemstone-rs.codegen BookingDraft Object
```

From a checkout:

```
cargo run -p gemstone-rs --example codegen_discover
cargo run -p gemstone-rs --example codegen_discover_mapping
```

Preview without writing:

```
gemstone-rs codegen preview gemstone-rs.codegen
```

Show a generated diff:

```
gemstone-rs codegen diff gemstone-rs.codegen
```

Check freshness in CI:

```
gemstone-rs codegen check gemstone-rs.codegen
```

Write generated wrappers:

```
gemstone-rs codegen generate gemstone-rs.codegen
```

Generated Shape

For:

```
method = Object>>printString | return=String | doc=Return the receiver printString.
```

The generated wrapper includes:

```
pub fn print_string(&mut self) -> Result<String> {
    let value = self.session.perform(self.oop, "printString", &[])?;
    match value {
        Value::String(value) => Ok(value),
        Value::Oop(oop) => self.session.fetch_string(oop),
        other => Err(Error::UnexpectedType {
            expected: "String",
            actual: format!("{other:?}"),
        }),
    }
}
```

Generated files are meant to be checked in. That makes diffs, reviews, and editor indexing predictable.

Generated Wrapper App

The repository includes a small live example that imports the checked-in generated wrapper and calls `Object>>printString` on a small integer:

```
cargo run -p gemstone-rs --example generated_wrapper_app
```

Expected output:

```
generated wrapper printString: 7
```

The example uses the generated `Object::from_oop` constructor:

```
#[path = "../../examples/codegen/generated/gemstone_wrappers.rs"]
mod gemstone_wrappers;

use gemstone_rs::{Config, Session};
```

```
fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env())?;
    let oop = session.smallint_oop(7);
    let mut object = gemstone_wrappers::Object::from_oop(&mut session, oop);
    println!("{}", object.print_string()?);
    Ok(())
}
```

Generated Object Mapping

The generated file can also include Rust data structs that implement `BridgeMapped`. Run:

```
cargo run -p gemstone-rs --example generated_mapping_app
```

Expected output:

```
generated mapped payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP",
tags: ["priority", "demo"] }
```

The config-driven mapping emits a struct like:

```
#[derive(Clone, Debug, Eq, PartialEq)]
pub struct BookingDraft {
    pub name: String,
    pub amount: i64,
    pub currency: String,
    pub tags: Vec<String>,
}
```

and an implementation of `BridgeMapped` so it works with:

```
bridge_root.put_mapped("GeneratedBookingDraft", &draft)?;
let loaded: BookingDraft = bridge_root.get_mapped("GeneratedBookingDraft")?;
```

VS Code Workflow

The `gemstone-rs Workbench` extension exposes the same flow in the GemStone RS sidebar:

- Discover from Live Stone
- Generate Mapping Config
- Preview BridgeRoot
- Run Generated Mapping Example

- Preview Wrappers
- Diff Generated Output
- Check Freshness
- Generate Wrappers
- Open Codegen Docs

Generate Wrappers shows the diff first and asks before writing when output would change.

This PDF was generated locally from docs/. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.